Today's agenda

↳ Patterns

AlgoPrep

Q) Print N "*" in a single row.

Ex: N=4 → * * * *

N iteration ←

```
for (int i=1; i<=N; i++){
        System.out.print("* ");
}
```

AlgoPrep

Q) Given Integer N, Print Square of N*N using "*".

Ex: N=4

```
* * * *
* * * *
* * * *
* * * *
```

N=5

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

N=3

How many rows

Printing in one row

```
for (int i=1; i<=N; i++){
    for int j=1; j<=N; j++)
        System.out.print("* ");
    }
    System.out.println();
}
```

```
* * *
* * *
* * *
```

i	i<=N
1	t
2	t
3	t
4	f → exit

j	j<=N
1	t
2	t
3	t
4	f

j	j<=N
1	t
2	t
3	t
4	f

j	j<=N
1	t
2	t
3	t
4	f

4 f
↳ exit

AlgoPrep

Q) Pattern 1:

↳ Print the triangle Pattern.

N=2:

```
*
* *
```

N=4:

```
*
* *
* * *
* * * *
```

+1 +1 +1

N=4

nst = ~~1 2~~ 3 4 5

```
int nst = 1;
for (int i=1; i<=N; i++){
    for (int j=1; j<=nst; j++){
        System.out.print("*");
    }
    nst++;
    System.out.println();
}
```

Print star

Prep for next line

exit

i	i<=N	j	j<=nst
1	t	1	t
		2	f → exit
2	t	1	t
		2	t
		3	f → exit
3	t	1	t
		2	t
		3	t
		4	f → exit
4	t	1	t
		2	t
		3	t
		4	t
5	→ exit	5	

```
*
* *
* * *
* * * *
```

→

Q) Pattern 2

↳ Point the following Pattern.

N = 3

```
_ * _
* * *
_ * _
```

N = 5

```
        _ _ * _ _
+2      _ * * * _
+2      * * * * *
-2      _ * * * _
-2      _ _ * _ _
```

N = 7

```
ns+     _ _ _ * _ _ _     nsD
+2      _ _ * * * _ _     -1
+2      _ * * * * * _     -1
+2      * * * * * * *     -1
-2      _ * * * * * _     +1
-2      _ _ * * * _ _     +1
-2      _ _ _ * _ _ _     +1
```

N = 6 → incorrect input

N=5

```
- - * * *
- * * * *
* * * * *
- * * * *
- - * * *
```

```
* *
```

```
int nst = 1;
int nsp = N/2;
```

first row

```
for (int i=1; i<=N; i++){

    for (int j=1; j<=nsp; j++){
        System.out.print(" ");
    }

    for (int k=1; k<=nst; k++){
        System.out.print("* ");
    }

    if (i<=N/2){
        nst = nst+2;
        nsp = nsp-1;
    }
    else {
        nst = nst-2;
        nsp = nsp+1;
    }

    System.out.println();
}
```

print for row

prep for next line

AlgoPrep

$n=5$

i=1 _ _ * * *
2 _ * * * *
3 * * * * *
4 _ * * * *
5 _ _ * * *

nsp = ~~2~~ ~~1~~ 0 nst = ~~1~~ ~~3~~ 5

```
int nst = 1;
int nsp = n/2;
for (int i=1; i<=n; i++){
    for (int j=1; j<=nsp; j++){
        System.out.print(" ");
    }
    for (int k=1; k<=nst; k++){
        System.out.print("*");
    }
    if (i<=n/2){
        nst = nst+2;
        nsp = nsp-1;
    }
    else {
        nst = nst-2;
        nsp = nsp+1;
    }
    System.out.println();
}
```

first row

Print for row

Prep for next line

$n=5$

i	i<=n	i<=n/2	
1	+	+	
2	+	2<=5/2 +	
3	+	3<=5/2 f	nsp=1, nst=3
4	+	4<=5/2 f	nsp=2, nst=1
5	+	5<=5/2 f	nsp=3, nst=-1
6	f ↳ exit		

_ _ *
_ * * *
* * * * *
_ * * *
↳ _ _ *

Break till 9:36 pm

Q) Pattern 3

↳ Print the following Pattern.

N = 5 :

```
* * * _ * * *
* * _ _ _ * *
* _ _ _ _ _ *
* * _ _ _ * *
* * * _ * * *
```

$$\frac{N+2}{2} \Rightarrow \frac{N}{2} + 1$$

N = 7

```
1   * * * * _ * * * *
2 -1 * * * _ _ _ * * *   +2
3 -1 * * _ _ _ _ _ * *   +2
4 -1 * _ _ _ _ _ _ _ *   +2
5 +1 * * _ _ _ _ _ * *   -2
6 +1 * * * _ _ _ * * *   -2
7 +1 * * * * _ * * * *   -2
```

AlgoPrep

// Pseudo code

```
int nsp = 1;
int nst = N/2 + 1;
for (int i=1; i<=N; i++){

    for (int j=1; j<=nst; j++){
        System.out.print("* ");
    }

    for (int k=1; k<=nsp; k++){
        System.out.print("  ");
    }

    for (int j=1; j<=nst; j++){
        System.out.print("* ");
    }

    if (i<= N/2){
        nst = nst-1;
        nsp = nsp+2;
    }
    else {
        nst = nst +1;
        nsp = nsp-2;
    }

    System.out.println();
}
```

AlgoPrep

```
int nsP = 1;
int nst = n/2 + 1;
for (int i=1; i<=N; i++){
    for (int j=1; j<=nst; j++){
        System.out.print("*");
    }
    for (int k=1; k<=nsP; k++){
        System.out.print(" ");
    }
    for (int j=1; j<=nst; j++){
        System.out.print("*");
    }
    if (i<=n/2){
        nst = nst-1;
        nsP = nsP+2;
    }
    else {
        nst = nst+1;
        nsP = nsP-2;
    }
    System.out.println();
}
```

```
***___***
**_____**
*_____*
**_____**
***___***
```

N=5:

```
1 * * * _ * * *
2 * * _ _ _ * *
3 * _ _ _ _ _ *
4 * * _ _ _ * *
5 * * * _ * * *
```

nsP = 1 nst = 3

i	i<=N	i<=n/2	
1	+	+	→ nsP=3, → nst=2
2	+	2<=5 +	→ nsP=5, → nst=1
3	+	3<=5 +	→ nsP=3, → nst=2
4			

AlgoPrep

Q) Pattern 4:

↳ Print the triangle Pattern with number

N=2:

```
1
2 3
```

N=4:

```
1
2 3
4 5 6
7 8 9 10
```

```
int val=1;
int nst = 1;
for (int i=1; i<=N; i++){

    for (int j=1; j<=nst; j++){
        System.out.println(val);
        val++;
    }

    nst ++;
    System.out.println();
}
```

AlgoPrep

listening to class

Passive learning

↓ reduces learning time

+

Active learning

AlgoPrep